

项目中的 Utils 工具详解

项目中的 Utils 工具详解

一、Utils 目录概述

项目中 `Utils`（工具）目录目前仅包含 `KeyboardExtension.swift` 一个文件，核心作用是**简化 iOS 应用中键盘的管理操作**，提供“键盘显示 / 隐藏”的便捷控制方法，避免重复编写基础逻辑，提升开发效率。

二、KeyboardExtension.swift 详解

`KeyboardExtension.swift` 是 SwiftUI 的扩展文件，通过自定义修饰符与方法，封装键盘管理核心功能，具体如下：

1. 功能 1：点击空白区域关闭键盘

通过 `hideKeyboardWhenTappedAround()` 方法，为视图添加“点击空白处自动收起键盘”的功能，解决用户“不知道如何关闭键盘”的痛点。

核心代码

```
func hideKeyboardWhenTappedAround() -> some View {
    modifier(DismissKeyboardOnTap())
}
// 自定义修饰符：处理点击空白区域的逻辑
private struct DismissKeyboardOnTap: ViewModifier {
    func body(content: Content) -> some View {
        content
        .onTapGesture {
            // 点击时调用“主动关闭键盘”方法
            UIApplication.shared.endEditing()
        }
    }
}
```

```
}  
}
```

功能说明

- 用户点击输入框弹出键盘后，点击视图内任何“非输入区域”（如空白背景、按钮），键盘会自动收起；
- 适用场景：包含输入框的表单页面（如登录页、注册页、任务编辑页）。

2. 功能 2：视图出现时设置键盘隐藏监听

通过 `hideKeyboardOnScroll()` 方法，为视图添加“应用进入后台时自动关闭键盘”的监听，避免用户返回应用时键盘仍停留的问题。

核心代码

```
func hideKeyboardOnScroll() -> some View {  
    modifier(DismissKeyboardOnScroll())  
}  
// 自定义修饰符：监听应用状态变化  
private struct DismissKeyboardOnScroll: ViewModifier {  
    @State private var notificationObserver: NSObjectProtocol?  
  
    func body(content: Content) -> some View {  
        content  
        .onAppear {  
            // 注册“应用进入后台”通知监听  
            notificationObserver = NotificationCenter.default.addObserver(  
                forName: UIApplication.willResignActiveNotification,  
                object: nil,  
                queue: .main  
            ) { _ in  
                // 应用进入后台时，关闭键盘  
                UIApplication.shared.endEditing()  
            }  
        }  
        .onDisappear {  
            // 视图消失时，移除通知监听（避免内存泄漏）  
            notificationObserver?.removeObserver(notificationObserver)  
        }  
    }  
}
```

```

        if let observer = notificationObserver {
            NotificationCenter.default.removeObserver(observer)
        }
    }
}
}

```

功能说明

- 视图出现时，自动监听 `UIApplication.willResignActiveNotification`（应用即将进入非活跃状态，如切换到其他应用、点击 Home 键）；
- 收到通知后，自动关闭键盘；视图消失时移除监听，避免内存泄漏；
- 适用场景：所有包含输入框的视图，尤其适合“用户可能频繁切换应用”的场景。

3. 功能 3：主动关闭键盘的方法

为 `UIApplication` 添加 `endEditing()` 扩展方法，支持在代码任意位置“主动触发键盘关闭”，无需复杂逻辑。

核心代码

```

extension UIApplication {
    func endEditing() {
        // 发送“放弃第一响应者”事件，触发键盘关闭
        sendAction(#selector(UIResponder.resignFirstResponder), to: nil, from: nil, for: nil)
    }
}

```

功能说明

- `resignFirstResponder`：让当前“第一响应者”（如输入框）放弃响应状态，从而关闭键盘；
- 适用场景：数据提交后（如点击“登录”“保存”按钮）、用户完成输入后，主动关闭键盘。

三、如何使用这些工具

所有功能均以“视图修饰符”或“类方法”形式提供，使用方式简洁，仅需 1 行代码即可集成：

1. 使用“点击空白区域关闭键盘”

在目标视图末尾添加 `.hideKeyboardWhenTappedAround()` 修饰符：

```
struct YourView: View {
  @State private var text = "" // 输入框绑定的状态

  var body: some View {
    VStack(spacing: 16) {
      // 输入框
      TextField("请输入内容", text: $text)
        .padding()
        .background(Color.gray.opacity(0.2))
        .cornerRadius(8)

      // 其他视图组件（如按钮、文本）
      Text("点击空白处关闭键盘")
        .foregroundColor(.gray)
    }
    .padding()
    .hideKeyboardWhenTappedAround() // 添加修饰符，启用功能
  }
}
```

2. 使用“视图出现时设置键盘隐藏监听”

在目标视图末尾添加 `.hideKeyboardOnScroll()` 修饰符：

```
struct YourView: View {
  @State private var text = ""

  var body: some View {
    VStack {
      TextField("请输入内容", text: $text)
        .padding()
        .background(Color.gray.opacity(0.2))
    }
  }
}
```

```
        .cornerRadius(8)
    }
    .padding()
    .hideKeyboardOnScroll() // 添加修饰符，启用监听
}
}
```

3. 使用 “主动关闭键盘”

在需要关闭键盘的逻辑中（如按钮点击、数据提交），调用 `UIApplication.shared.endEditing()`：

```
// 示例1：按钮点击事件
Button("关闭键盘") {
    UIApplication.shared.endEditing()
}
// 示例2：数据提交后
func submitData() {
    // 1. 处理数据提交逻辑（如网络请求）
    print("提交数据：\(text)")

    // 2. 主动关闭键盘
    UIApplication.shared.endEditing()
}
```

四、实际场景应用例子

以“登录页面”为例，集成所有键盘管理功能，提升用户体验：

```
struct LoginView: View {
    // 输入框绑定的状态
    @State private var username = ""
    @State private var password = ""

    var body: some View {
        VStack(spacing: 20) {
            // 用户名输入框
```

```

TextField("用户名", text: $username)
    .padding()
    .background(Color.gray.opacity(0.2))
    .cornerRadius(8)
    .keyboardType(.emailAddress) // 弹出邮箱键盘

// 密码输入框
SecureField("密码", text: $password)
    .padding()
    .background(Color.gray.opacity(0.2))
    .cornerRadius(8)

// 登录按钮
Button("登录") {
    submitLogin() // 点击触发登录逻辑
}
    .padding()
    .background(Color.themePurple)
    .foregroundColor(.white)
    .cornerRadius(8)
    .disabled(username.isEmpty || password.isEmpty) // 输入为空时禁用
}
    .padding()
    .hideKeyboardWhenTappedAround() // 点击空白处关闭键盘
    .hideKeyboardOnScroll() // 应用进入后台时关闭键盘
}

// 登录逻辑
private func submitLogin() {
    // 1. 主动关闭键盘
    UIApplication.shared.endEditing()

    // 2. 处理登录请求（如调用AuthService）
    Task {
        do {
            try await AuthService.shared.login(username: username, password: password)
            // 登录成功：跳转主界面
        } catch {
            // 登录失败：显示错误提示
            print("登录失败：\(error.localizedDescription)")
        }
    }
}

```

```
}  
}  
}
```

五、总结

Utils 目录中的 KeyboardExtension.swift 虽功能单一，但精准解决了 iOS 开发中“键盘管理”的高频痛点，核心优势如下：

- 1. 使用简洁：**所有功能均通过“1 行修饰符”或“1 行方法调用”集成，无需编写复杂逻辑；
- 2. 体验友好：**覆盖“用户主动关闭”“自动关闭”等场景，避免用户因“无法关闭键盘”产生困惑；
- 3. 通用性强：**适配所有包含输入框的视图，可在登录页、注册页、任务编辑页等场景复用；
- 4. 性能安全：**通知监听在视图消失时自动移除，避免内存泄漏。

若后续需扩展工具目录，可参考此文件的“简洁封装”思路，将其他通用功能（如日期格式化、字符串验证）以“扩展”或“工具类”形式封装，进一步提升项目可维护性。

（注：文档部分内容可能由 AI 生成）